

Web Applications I 2025/26 – Exam #2 (deadline 2026-07-09 at 23:59)

“Battleship”

⚠️ **PRELIMINARY VERSION** – Leave any questions as **COMMENTS** and do not delete or resolve any existing comments. The final version will be published on 2026-06-29.

Design and implement a web application for a single-player game inspired by the board game “Battleship”.

In the game, the application positions a set of ships of different sizes on an $N \times N$ grid. Each ship is represented by a consecutive number of cells (at least 2) positioned in a horizontal or vertical direction. The player must guess the position of the ships by sending torpedoes in the different cells of the grid. The player plays against the server in a set of matches.

Each match has a difficulty level that may be set to “Easy”, “Intermediate”, or “Hard”. The level of difficulty influences the size of the grid (N), and the number and sizes of the ships positioned by the server. Each difficulty level must respect the following constraints:

- Grid size: from 5×5 to 15×15
- Number of ships: from 4 to 20
- Ship size: from 2 to 5

At the beginning of a match, the server secretly positions a fixed number of ships (depending on the difficulty level), of different sizes. For example, the Easy game might be played on a 5×5 grid ($N=5$), and the server positions 2 ships of size 2, 1 ship of size 3, and 1 ship of size 4 (NOTE: these numbers are just an example, the student should define its own criteria). Ships are positioned in random positions in the grid, in random orientation (vertically or horizontally), with the constraint that they must not be adjacent (i.e., between any two ships we must always have one cell of free water).

During a match, the application shows how many ships are in the grid, and their sizes. The player has a fixed number of torpedoes, that depends on the difficulty level (each project must define how many initial torpedoes are available for each difficulty level). When launching a torpedo, the player selects one cell from the whole grid. The result of a launch may be:

- water: the cell does not contain any ship
- hit: the cell contains one part of a ship
- hit and sunk: the cell contains the last remaining part of a ship.

In case of a ‘water’ launch, the player loses a torpedo, in case of ‘hit’ or ‘hit and sunk’, he/she keeps all current torpedoes. The grid should graphically reflect the result of all launches.

The match ends when the player runs out of torpedoes (match lost) or when all the ships have been sunk (match won). At the end of the game, the position of all ships is shown in the grid. The application keeps track, for each player and for each difficulty level, of the following statistics: the number of matches played, won, lost, and the percentage of won matches. Statistics **for all users** are then shown in a dedicated page, **accessible without login**.

The game may be played in two different ways:

- **Casual:** an anonymous user (i.e., not logged in) plays a single match, and no result is saved.
- **Tournament:** several logged-in users compete, ~~simultaneously,~~ on the same grid. In tournament mode, the first logged-in player starts a new tournament, and the application provides a textual code for that tournament **and immediately starts the match**. Other users may enter an existing tournament by inserting the same code: this will start a new match (on an empty grid) where the ship positions, grid size, and difficulty are the same (the tournament code thus defines a sort of .

The organization of these specifications into different screens, and possibly different routes, is left to the student.

To avoid cheating, the positions of the ships must never be transferred to the client until the end of each match.

Project requirements

- The application architecture and source code must be developed by adopting the best practices in software development, in particular, those relevant to single-page applications (SPA) using React and HTTP APIs. APIs should be carefully protected and the front-end should not receive unnecessary information.
- The application should be designed for a desktop browser. Responsiveness for mobile devices is not required nor evaluated.
- The project must be implemented as a React 19 application that interacts with an HTTP API implemented in Node+Express. The Node version must be the one used during the course (24.x, LTS). The database must be stored in a SQLite file. The programming language must be JavaScript.
- The communication between client and server must follow the “two servers” pattern, by properly configuring CORS, and React must run in “development” mode with Strict Mode activated.
- The evaluation of the project will be carried out by navigating the application. Neither the behavior of the “refresh” button, nor the manual entering of a URL (except /) will be tested, and their behavior is undefined. Also, the application should never “reload” itself as a consequence of normal user operations.
- The root directory of the project must contain a README.md file and have two subdirectories (client and server). The project must be started by running the two commands: “cd server; nodemon index.js” and “cd client; npm run dev”. A template for the project directories is already available in the exam repository. You may assume that nodemon is globally installed. No other global modules will be available.
- The whole project must be submitted on GitHub, on the same repository created by GitHub Classroom.
- The project **must not include** the `node_modules` directories. They will be re-created by running the “`npm install`” command right after “`git clone`”.
- The project may use popular and commonly adopted libraries (for example, `day.js`, `react-bootstrap`, etc.), if applicable and useful. All required libraries must be correctly declared in the `package.json` file, so that the `npm install` command might install them.

- User authentication (login and logout) and API access must be implemented with `Passport.js` and session cookies. The credentials should be stored in an encrypted and salted form. The user registration procedure is not requested nor evaluated.

Quality requirements

In addition to the implementation of the required application functionality, the following quality requirements will be evaluated:

- Database design and organization.
- Design of the HTTP APIs.
- Organization of React components and routes.
- Correct usage of React patterns (functional behavior, hooks, state, context, and effects). Avoiding direct manipulation of the DOM is included in these rules.
- Code clarity.
- Absence of errors (and warnings) in the client console (except those caused by errors in the imported libraries).
- Absence of application crashes or unhandled exceptions.
- Essential data validation (in Express and React).
- Basic usability and user-friendliness.
- Commit history in the GitHub repository
- Originality of the solution.

NOTE: In the evaluation of the project, during the oral discussion, it is expected that the project has been fully developed by the candidate student, who should therefore possess detailed knowledge about the code, and especially about the design decisions and the adopted functions or solutions. It is not forbidden to collaborate with other students (but not sharing whole parts of the project), nor to utilize AI coding assistance, but the student still has 100% responsibility for the knowledge, understanding and capacity to explain the code.

Database requirements

The project database must be designed by the student and must be pre-populated (seeded) with the initial data for supporting multiple gameplays .

In addition, the database must contain at least 3 users, of which 2 users must have already played some games (won and lost).

Contents of the README.md file

The README.md file must contain the following information (a template is available in the project repository). Generally, each information item should take no more than 2-3 lines.

1. Server-side:
 - a. A list of the HTTP APIs offered by the server, with a short description of the parameters and the exchanged objects.
 - b. A list of the database tables, with their purpose.
2. Client-side:
 - a. A list of 'routes' for the React application, with a short description of the purpose of each route.

- b. A list of the main React components.
- 3. Overall:
 - a. One screenshot of the application, showing what happens during a match. The screenshot must be embedded in the README by linking the image committed in the project repository.
 - b. Usernames and passwords of the registered users.
 - c. The possible usage of AI during the development of the project, describing for what purpose AI has been used and how their output has been verified and/or adapted. If you didn't use any AI, please state it.

Submission procedure

To correctly submit the project, you must:

- **Be enrolled** in the exam call.
- Use the provided **link to join the classroom** on GitHub Classroom (i.e., correctly **associate your GitHub username with your student ID**) and **accept the assignment**.
(<https://classroom.github.com/a/5L9YsWm8>)
- **Push the project** into the **main branch** of the repository created for you by GitHub Classroom. The last commit (the one you wish to be evaluated) must be **tagged** with the tag **final** (note: final is all-lowercase with no spaces, and it is a git 'tag', not a 'commit message').

Note: to tag a commit, you may use (from the terminal) the following commands:

```
# ensure the latest version is committed
git commit -m "...comment..."
git push

# add the 'final' tag and push it
git tag final
git push origin --tags
```

Alternatively, you may insert the tag from GitHub's web interface (follow the link 'Create a new release').

To test your submission, these are the exact commands that the teachers will use to download and run the project. You may wish to test them on a clean directory:

```
git clone ...yourCloneURL...
cd ...yourProjectDir...
git pull origin main # just in case the default branch is not main
git checkout -b evaluation final # check out the version tagged with
'final' and create a new branch 'evaluation'
(cd client ; npm install; npm run dev)
(cd server ; npm install; nodemon index.js)
```

Ensure that all the needed packages are downloaded by the npm install commands. Be careful: if some packages are installed globally on your computer, they might not be listed as dependencies. Always check it in a clean installation.

Be aware that Linux is case-sensitive for file names, while Windows and macOS are not. Double-check the case of import statements and all file names.